

Lab 11 - Services, Endpoints and Cluster DNS

Field	Value
Lab ID	Lab 11
Day / Topic	Day 3 Services and Networking
Duration	50 minutes
Namespace	kcna-lab11
Mapping	LO3 A2 K2, K4
Cluster	Single-node kind cluster, Kubernetes v1.30 or later

1. Objective

By the end of this lab you will be able to:

1. Explain a Service as a selector plus a port map, and the EndpointSlice as the *product* that the endpoints controller derives from that selector.
2. Distinguish port, targetPort and a named container port, and change the Pod listener without touching the Service.
3. Resolve a Service through cluster DNS in all four name forms (<svc>, <svc>.<ns>, <svc>.<ns>.svc, <svc>.<ns>.svc.cluster.local) and explain the search list in the Pod's /etc/resolv.conf that makes the short forms work.
4. Read kubectl get endpointslices as the primary Service-backend diagnostic, and recognise v1 Endpoints as the deprecated predecessor.
5. Diagnose a Service that has zero endpoints because of a selector typo, and name the exact symptom the client sees.

2. Prerequisites

- A running single-node kind cluster and a kubectl whose minor version is within one of the cluster (kubectl version).
- Labs 01-10 completed (Pods, labels, selectors, Deployments).
- Your shell's working directory is the lab folder:

```
cd courseware/labs/lab-11-services-endpoints
pwd
```

Expected output (your absolute prefix will differ):

```
/.../TGS-2023039343-Kubernetes and Cloud Native Associate (KCNA) Training/courseware/labs/lab-11-services-endpoints
```

- Confirm the cluster answers and CoreDNS is running - every DNS step below depends

on it:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

Expected output (names and ages will differ):

NAME	READY	STATUS	RESTARTS	AGE
coredns-668d6bf9bc-4nzt8	1/1	Running	0	41m
coredns-668d6bf9bc-r7wqk	1/1	Running	0	41m

If this returns No resources found, stop. Without CoreDNS, steps 6-8 cannot work

and no amount of Service debugging will help.

3. Scenario

Meridian Freight Pte Ltd runs a container-shipping booking platform. The quote-api service prices a shipment for a given lane and container type. It is being moved off a fixed VM address onto Kubernetes.

The VM version listened on :8080 and other teams hard-coded `http://10.20.4.11:8080` into their configuration. Every time the VM was rebuilt the IP changed and three downstream teams broke. Your job as platform engineer is to give quote-api a stable name - `quote-api.kcna-lab11.svc.cluster.local` - that keeps working while the Pods behind it are replaced, scaled and rescheduled, and to be able to prove *which* Pods a caller is actually reaching.

4. Step-by-step procedure

Step 1 - Create the namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

Expected output:

```
namespace/kcna-lab11 created
```

Confirm it exists and is Active:

```
kubectl get namespace kcna-lab11
```

Expected output:

NAME	STATUS	ACTIVE	AGE
kcna-lab11	Active	5s	

Note: on some releases the column header set is NAME STATUS AGE. The value that matters is Active.

Step 2 - Understand where the Pod content comes from

The quote-api Pods serve a static page held in a ConfigMap. That ConfigMap is the declarative form of the file in data/. Look at the source file first:

```
cat data/quote-api-index.html
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>
<p>service: quote-api</p>
<p>namespace: kcna-lab11</p>
<p>listener: container port 8080 (named "http")</p>
<p>build: 2026.09.05-lab11</p>
</body>
</html>
```

manifests/10-configmap-quote-content.yaml is exactly what this command produces - generate it yourself to prove it, without applying anything:

```
kubectl create configmap quote-api-content \
  --from-file=index.html=data/quote-api-index.html \
  -n kcna-lab11 --dry-run=client -o yaml | head -8
```

Expected output:

```
apiVersion: v1
data:
  index.html: |
    <!DOCTYPE html>
    <html lang="en">
    <head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
    <body>
    <h1>Meridian Freight Pte Ltd</h1>
```

Now apply the checked-in version (it also carries the nginx server block that moves the listener to port 8080):

```
kubectl apply -f manifests/10-configmap-quote-content.yaml
```

Expected output:

```
configmap/quote-api-content created
configmap/quote-api-nginx created
```

Step 3 - Deploy the workload

```
kubectl apply -f manifests/20-deployment-quote-api.yaml
```

Expected output:

```
deployment.apps/quote-api created
```

Wait for all three replicas to become Ready - readiness, not merely Running, is what puts a Pod into the EndpointSlice:

```
kubectl rollout status deployment/quote-api -n kcna-lab11 --timeout=120s
```

Expected output:

```
Waiting for deployment "quote-api" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "quote-api" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "quote-api" rollout to finish: 2 of 3 updated replicas are available...
deployment "quote-api" successfully rolled out
```

List the Pods with their IPs - these are the addresses the Service will collect:

```
kubectl get pods -n kcna-lab11 -l app=quote-api -o wide
```

Expected output (Pod name suffixes and IPs are allocated at runtime and will differ on your cluster):

NAME	NOMINATED NODE	READY	STATUS	RESTARTS	AGE	IP	NODE
quote-api-6c9b4f7d55-7lqm2	<none>	1/1	Running	0	38s	10.244.0.14	kind-control-plane
quote-api-6c9b4f7d55-jd8vx	<none>	1/1	Running	0	38s	10.244.0.15	kind-control-plane
quote-api-6c9b4f7d55-x2t9k	<none>	1/1	Running	0	38s	10.244.0.16	kind-control-plane

Record these three Pod IPs. Step 5 shows the identical set appearing inside the

EndpointSlice - that is the whole point of this lab.

Step 4 - Create the Service and read its three port fields

```
kubectl apply -f manifests/30-service-quote-api.yaml
```

Expected output:

```
service/quote-api created
```

```
kubectl get service quote-api -n kcna-lab11
```

Expected output (the CLUSTER-IP is allocated from the cluster's service CIDR and will differ):

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
quote-api	ClusterIP	10.96.114.23	<none>	80/TCP	6s

Read the port fields deliberately:

Field	Value here	Meaning
spec.ports[0].port	80	The port the Service listens on. This is what clients dial: quote-api:80.
spec.ports[0].targetPort	http	The port on the Pod. Because it is a <i>*name*</i> , kubelet resolves it against the container's ports[].name.

Field	Value here	Meaning
container ports[0].containerPort	8080	The port the process actually binds.
spec.ports[0].name	http	The Service port's own name. Required once a Service has more than one port; also what an Ingress backend.service.port.name refers to.

Confirm the name resolution really happened by asking the API server for the resolved target:

```
kubectl get service quote-api -n kcna-lab11 -o jsonpath='{.spec.ports[0].targetPort}{"\n"}'
```

Expected output:

```
http
```

The Service object keeps the **name**; the resolution to 8080 happens per-Pod, in the EndpointSlice. You will see the number in the next step.

Step 5 - The EndpointSlice: what the selector produced

`kubectl get endpointslices` is the current, supported way to inspect Service backends. (v1 Endpoints was deprecated in Kubernetes v1.33; it is still served for compatibility - see Step 5b.)

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api
```

Expected output (the slice name gets a random suffix):

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-t8hkb	IPv4	8080	10.244.0.14, 10.244.0.15, 10.244.0.16	45s

Two things to notice:

- PORTS is 8080, not 80. The named targetPort: http was resolved to the real container port when the slice was written.
- ENDPOINTS is exactly the three Pod IPs from Step 3.

Print the same information in a form you can compare against Step 3:

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api \
-o jsonpath='{range .items[*].endpoints[*]}.{addresses[0]}{"\t"}{.conditions.ready}{"\t"}{.targetRef.name}{"\n"}{end}'
```

Expected output:

```
10.244.0.14■true■quote-api-6c9b4f7d55-7lqm2
10.244.0.15■true■quote-api-6c9b4f7d55-jd8vx
10.244.0.16■true■quote-api-6c9b4f7d55-x2t9k
```

`conditions.ready` is the readiness probe's verdict. A Pod that is Running but not Ready still appears in the slice with `ready: false`, and kube-proxy will not send it traffic. This is why "the Pod is running but the Service is broken" is such a common incident.

Step 5b - The deprecated Endpoints view

```
kubectl get endpoints quote-api -n kcna-lab11
```

Expected output:

NAME	ENDPOINTS	AGE
quote-api	10.244.0.14:8080,10.244.0.15:8080,10.244.0.16:8080	70s

Same data, older shape. The v1 Endpoints API was deprecated in Kubernetes v1.33 because a single object could not scale past a few thousand addresses and every update rewrote the whole list. EndpointSlice shards them (default 100 endpoints per slice) and adds topology and per-endpoint conditions. For the KCNA exam and for real diagnostics, reach for endpointslices.

Step 6 - Start a client Pod and read its resolver configuration

```
kubectl apply -f manifests/40-pod-dns-client.yaml
```

Expected output:

```
pod/dns-client created
```

```
kubectl wait --for=condition=Ready pod/dns-client -n kcna-lab11 --timeout=60s
```

Expected output:

```
pod/dns-client condition met
```

Before resolving anything, look at **why** short names work:

```
kubectl exec -n kcna-lab11 dns-client -- cat /etc/resolv.conf
```

Expected output:

```
search kcna-lab11.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5
```

Read it line by line:

- nameserver 10.96.0.10 - the ClusterIP of the kube-dns Service in

kube-system (the Service is named kube-dns even though CoreDNS serves it). kubelet injects this into every Pod with dnsPolicy: ClusterFirst, the default.

- search ... - appended in order to any name with fewer than ndots dots. This is

what turns quote-api into quote-api.kcna-lab11.svc.cluster.local on the first try.

- options ndots:5 - a name containing fewer than 5 dots is treated as **relative**

and pushed through the search list first. quote-api.kcna-lab11.svc.cluster.local has 4 dots, so even the "FQDN" is walked through the search list before being tried absolute - the classic source of extra DNS lookups. Add a trailing dot to force an absolute query.

Step 7 - Resolve the Service in all four name forms

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api.kcna-lab11.svc.cluster.local
```

Expected output:

```
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: quote-api.kcna-lab11.svc.cluster.local
Address: 10.96.114.23
```

Honest note on BusyBox nslookup: BusyBox also issues an AAAA (IPv6) query. On

an IPv4-only kind cluster you will often see an extra

*** Can't find quote-api.kcna-lab11.svc.cluster.local: No answer line before or

after the answer above, and the command may exit non-zero. That is not a

failure of your Service - it is the missing IPv6 record. Judge the result by the

Address: line.

Now the shorter forms, from a Pod in the same namespace:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api
```

Expected output (same ClusterIP, reached via the first search-path entry):

```
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: quote-api.kcna-lab11.svc.cluster.local
Address: 10.96.114.23
```

Note that the *answer* is the FQDN even though you asked for the short name - proof the search path was applied.

Now the two partial forms. Use ping rather than nslookup here, and read only the resolution shown in ping's first line:

```
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11
```

Expected output (the ClusterIP will differ; the packet loss is expected and irrelevant):

```
PING quote-api.kcna-lab11 (10.96.114.23): 56 data bytes

--- quote-api.kcna-lab11 ping statistics ---
1 packets transmitted, 0 packets received, 100% packet loss
```

```
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11.svc
```

Expected output:

```
PING quote-api.kcna-lab11.svc (10.96.114.23): 56 data bytes

--- quote-api.kcna-lab11.svc ping statistics ---
1 packets transmitted, 0 packets received, 100% packet loss
```

Two honest points here.

>

1. 0 packets received is correct. A ClusterIP is a kube-proxy DNAT rule, not a host - it is not obliged to answer ICMP. The line that matters is the resolved address in parentheses. Never use "ping fails" as evidence that a Service is down.
2. nslookup is the wrong tool for these two names. BusyBox nslookup sends the name it was given without walking /etc/resolv.conf's search list once the name already contains a dot, so nslookup quote-api.kcna-lab11 returns
 *** Can't find quote-api.kcna-lab11: No answer on a cluster where the name resolves perfectly well for real clients. ping, wget and any application using the standard resolver do apply the search list. verification/checks.sh therefore uses ping's resolution line, not nslookup, to test the fixtures.

Confirm point 2 for yourself with the tool that a real caller would use:

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api.kcna-lab11/ | head -1
```

Expected output:

```
<!DOCTYPE html>
```

Work through the fixture list in data/dns-fixtures.csv, which records every form and whether it should resolve:

```
cat data/dns-fixtures.csv
```

Expected output:

```
name,kind,resolves,note
quote-api,short,yes,resolved via the search path of the client Pod in the same namespace
quote-api.kcna-lab11,ns-qualified,yes,second search-path entry svc.cluster.local is appended
quote-api.kcna-lab11.svc,partial-fqdn,yes,cluster.local is appended from the search path
quote-api.kcna-lab11.svc.cluster.local,fqdn,yes,fully qualified - no search path needed
quote-api.default.svc.cluster.local,wrong-namespace,no,the Service does not exist in the default namespace
kubernetes.default.svc.cluster.local,control-plane,yes,the always-present API server Service
```

Prove the negative case - the same Service name in the wrong namespace:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api.default.svc.cluster.local
```

Expected output (non-zero exit code is correct here):

```
Server:10.96.0.10
Address:10.96.0.10:53

*** Can't find quote-api.default.svc.cluster.local: No answer

command terminated with exit code 1
```

The same negative with ping, which is what checks.sh uses:

```
kubectl exec -n kcna-lab11 dns-client -- ping -c 1 -W 1 quote-api.default.svc.cluster.local
```

Expected output:

```
ping: bad address 'quote-api.default.svc.cluster.local'
command terminated with exit code 1
```

bad address means resolution itself failed - there is no line with a resolved IP in parentheses. Contrast that with the partial names above, which resolved and then merely dropped ICMP.

A Service name is namespaced. Cross-namespace callers must use at least <svc>.<ns>; the bare name only works inside the owning namespace.

Step 8 - Actually use the Service

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- http://quote-api/
```

Expected output:

```
<!DOCTYPE html>
<html lang="en">
<head><meta charset="utf-8"><title>Meridian Freight - Quote API</title></head>
<body>
<h1>Meridian Freight Pte Ltd</h1>
<p>service: quote-api</p>
<p>namespace: kcna-lab11</p>
<p>listener: container port 8080 (named "http")</p>
<p>build: 2026.09.05-lab11</p>
</body>
</html>
```

The client dialled port 80 (the Service port, the default for http://) and was delivered to port 8080 in a Pod. That translation is the Service's job.

Now watch the endpoint set follow the Pod set. Scale down:

```
kubectl scale deployment/quote-api -n kcna-lab11 --replicas=1
```

Expected output:

```
deployment.apps/quote-api scaled
```

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api
```

Expected output (one address remains; the ClusterIP has not changed):

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-t8hkb	IPv4	8080	10.244.0.14	4m12s

Scale back up before continuing:

```
kubectl scale deployment/quote-api -n kcna-lab11 --replicas=3
kubectl rollout status deployment/quote-api -n kcna-lab11 --timeout=120s
```

Expected output:

```
deployment.apps/quote-api scaled
deployment "quote-api" successfully rolled out
```

The stable thing is the name and the ClusterIP. Everything behind it is disposable.

5. Verification

Run the automated checks:

```
bash verification/checks.sh
```

Expected output:

```
== Lab 11 verification: Services, Endpoints and Cluster DNS ==
PASS namespace kcna-lab11 exists
PASS ConfigMap quote-api-content matches data/quote-api-index.html byte-for-byte
PASS deployment quote-api has 3/3 ready replicas
PASS service quote-api is ClusterIP with port 80
PASS service quote-api targetPort is the named port "http"
PASS EndpointSlice resolved the named port to 8080
PASS EndpointSlice holds 3 ready addresses
PASS EndpointSlice addresses match the quote-api Pod IPs exactly
PASS dns fixture: quote-api resolves
PASS dns fixture: quote-api.kcna-lab11 resolves
PASS dns fixture: quote-api.kcna-lab11.svc resolves
PASS dns fixture: quote-api.kcna-lab11.svc.cluster.local resolves
PASS dns fixture: quote-api.default.svc.cluster.local correctly does NOT resolve
PASS dns fixture: kubernetes.default.svc.cluster.local resolves
PASS HTTP GET through the Service returns the Meridian Freight page
-- 15 passed, 0 failed --
```

See verification/expected-output.md for the full annotated reference output.

6. Failure injection - the selector typo

This is the single most common Service fault in production. You will create it deliberately.

6.1 Break it

```
kubectl apply -f manifests/50-service-broken-selector.yaml
```

Expected output:

```
service/quote-api-broken created
```

The API server accepted it without complaint. There is no validation that a Service selector matches anything - a Service with zero backends is a legal object.

6.2 Observe the symptom the caller sees

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api-broken/
```

Expected output (the ClusterIP will differ):

```
wget: can't connect to remote host (10.96.203.77): Connection refused
command terminated with exit code 1
```

Note carefully: DNS worked. The name resolved to a ClusterIP. Confirm it:

```
kubectl exec -n kcna-lab11 dns-client -- nslookup quote-api-broken
```

Expected output:

```
Server: 10.96.0.10
Address: 10.96.0.10:53

Name: quote-api-broken.kcna-lab11.svc.cluster.local
Address: 10.96.203.77
```

This is the diagnostic fork every engineer must be able to take:

- Name does not resolve → DNS / CoreDNS / wrong namespace / wrong Service name.
- Name resolves but the connection is refused → the Service exists but has **no endpoints**. In iptables mode kube-proxy installs a REJECT rule for a ClusterIP with an empty endpoint set, which is why you get an immediate ***Connection refused*** rather than a timeout.

6.3 Diagnose

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api-broken
```

Expected output:

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-broken-4zq6n	IPv4	<unset>	<unset>	40s

An empty slice. Now the classic confirmation:

```
kubectl describe service quote-api-broken -n kcna-lab11
```

Expected output (abridged; look at the Endpoints line):

```

Name:                quote-api-broken
Namespace:           kcna-lab11
Labels:              app=quote-api
                    kcna.tertiaryinfotech.com/intent=failure-injection
Selector:            app=quote-apis
Type:                ClusterIP
IP Family Policy:    SingleStack
IP Families:         IPv4
IP:                  10.96.203.77
IPs:                 10.96.203.77
Port:                http 80/TCP
TargetPort:          http/TCP
Endpoints:             

Session Affinity:    None
Events:              <none>

```

Endpoints: is empty and Selector: app=quote-apis is the culprit.

6.4 Prove the root cause

Ask the cluster what the selector actually matches. This is the decisive step - run the Service's own selector as a label query:

```
kubectl get pods -n kcna-lab11 -l app=quote-apis
```

Expected output:

```
No resources found in kcna-lab11 namespace.
```

Compare with the correct label:

```
kubectl get pods -n kcna-lab11 -l app=quote-api --no-headers | wc -l
```

Expected output:

```
3
```

Diagnosis: the Service selector app=quote-apis matches zero Pods. The Pods carry app=quote-api. The endpoints controller has nothing to put in the EndpointSlice, so kube-proxy has no destination to DNAT to and rejects the connection at the ClusterIP.

6.5 Fix and re-verify

Patch the selector to the correct label:

```
kubectl patch service quote-api-broken -n kcna-lab11 \
  --type=merge -p '{"spec":{"selector":{"app":"quote-api"}}}'
```

Expected output:

```
service/quote-api-broken patched
```

```
kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=quote-api-broken
```

Expected output:

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AGE
quote-api-broken-4zq6n	IPv4	8080	10.244.0.14,10.244.0.15,10.244.0.16	2m5s

```
kubectl exec -n kcna-lab11 dns-client -- wget -qO- --timeout=5 http://quote-api-broken/ | head -1
```

Expected output:

```
<!DOCTYPE html>
```

The endpoint set repopulated within a second of the patch, with no Pod restart. The Service was never "down" - it was simply pointing at nothing.

7. Troubleshooting

Symptom	Likely cause	Command that confirms it	Fix	
wget: can't connect to remote host (10.96.x.y): Connection refused and the name resolves	Service has zero endpoints: selector matches no Pod, or no Pod is Ready	kubectl get endpointslices -n kcna-lab11 -l kubernetes.io/service-name=<svc> shows no addresses	Correct spec.selector to match the Pod template labels, or fix the readiness probe	
nslookup: can't resolve 'quote-api' from a Pod in another namespace	Short names are namespaced; the search path of the *client* Pod is used	kubectl exec <pod> -- cat /etc/resolv.conf - first search entry is the client's namespace	Use quote-api.kcna-lab11 or the full FQDN	
nslookup quote-api.kcna-lab11 says No answer, but wget http://quote-api.kcna-lab11/ works	BusyBox nslookup does not walk the search list for a name that already contains a dot	kubectl exec dns-client -- ping -c 1 -W 1 quote-api.kcna-lab11 shows the resolved IP in its header	Diagnose with ping/wget, or query the full FQDN with nslookup. Nothing to fix on the cluster	
ping to a ClusterIP shows 100% packet loss	A ClusterIP is a kube-proxy DNAT rule, not a host; it need not answer ICMP	The PING <name> (<ip>) header still shows a resolved address	Test with TCP (wget), never with ICMP	
Connection times out instead of being refused	Not an empty-endpoint problem - a NetworkPolicy, a wrong port, or the app not listening on containerPort	kubectl get netpol -n kcna-lab11; kubectl exec <backend-pod> -- netstat -ltn	Align containerPort with the real listener; see Lab 16 for policy	
EndpointSlice lists an address with ready: false	Pod is Running but failing its readiness probe	kubectl describe pod <pod> -n kcna-lab11 → Readiness probe failed: events	Fix the probe path/port or the app's health handler	
Error: couldn't find port "http" on Pod start, or TargetPort: http never resolves	The named targetPort has no matching ports[].name in the container spec	kubectl get deploy quote-api -n kcna-lab11 -o jsonpath='{.spec.template.spec.containers[0].ports}'	Add name: http to the container port, or use the numeric targetPort: 8080	
kubectl get endpoints prints a deprecation notice or is missing on a future cluster	v1 Endpoints is deprecated as of Kubernetes v1.33	`kubectl api-resources \	grep -E 'endpoint`	Use kubectl get endpointslices

8. Cleanup

Delete only this lab's namespace. Every object you created lives inside it.

```
kubectl delete namespace kcna-lab11
```

Expected output:

```
namespace "kcna-lab11" deleted
```

Confirm:

```
kubectl get namespace kcna-lab11
```

Expected output:

```
Error from server (NotFound): namespaces "kcna-lab11" not found
```

Do not run an unscoped `kubectl delete svc --all` - without `-n kcna-lab11` that

targets your current namespace and can remove the Kubernetes Service.

9. What you learned

- A Service is two things: a stable virtual IP + DNS name, and a **label selector**.

It holds no list of Pods.

- The endpoints controller watches Pods, evaluates the selector, and writes an

EndpointSlice. That slice - not the Service - is the answer to "what is actually behind this name right now".

- Only Ready Pods are given `conditions.ready: true` and receive traffic.
- port (Service) → targetPort (Pod) is a translation, and using a named port

decouples the Service from the container's port number.

- Cluster DNS gives `<svc>.<ns>.svc.cluster.local`; the Pod's search path and

`ndots:5` make the shorter forms work within the client's namespace only.

- The signature of an empty Service is: DNS resolves, TCP is refused. Always run

the Service's selector as a `kubectl get pods -l query`.

- v1 Endpoints is deprecated (Kubernetes v1.33); `discovery.k8s.io/v1 EndpointSlice`

is the current API.

10. Further reading

- Service - <https://kubernetes.io/docs/concepts/services-networking/service/>
- EndpointSlices - <https://kubernetes.io/docs/concepts/services-networking/endpoint-slices/>
- DNS for Services and Pods - <https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/>
- Debug Services - <https://kubernetes.io/docs/tasks/debug/debug-application/debug-service/>
- Endpoints API deprecation - <https://kubernetes.io/blog/2025/04/24/endpoints-deprecation/>
- Virtual IPs and Service proxies - <https://kubernetes.io/docs/reference/networking/virtual-ips/>